

StabilityLoadingPlanner

Relatório Final de Projecto

Aplicação Android de Planeamento de Carga e Estabilidade de Navios

Lucas Alemão · Nº 15052

Engenharia Informática e de Computadores

Computação Móvel — 2025/2026

ENIDH — Escola Superior Náutica Infante D. Henrique

Junho de 2026

Índice

Índice	2
1. Introdução	4
1.1 Público-alvo	4
2. Processo de Desenvolvimento	4
2.1 Fase de Concept	4
2.2 Fase de Pre-Production	4
2.3 Fase de Production	5
2.4 Fase de Post-Production	5
3. Funcionalidades Implementadas	6
3.1 Funcionalidades Core (gratuitas)	6
3.2 Funcionalidade PRO (€1/mês)	6
3.3 Funcionalidade Extra — Inteligência Artificial	6
4. Arquitectura e Tecnologias	7
4.1 Stack Tecnológico	7
4.2 APIs Externas	7
4.3 Estratégia de Estado: Privado vs. Partilhado	8
5. Diagrama de Entidade-Associação e Esquema da Base de Dados	9
6. Diagrama de Classes UML	11
7. Wireframes e Mock-ups Iniciais	13
7.1 Nota sobre a fase de Mock-up	13
8. Mapa de Navegação	14
9. Capturas de Ecrã Finais	15
10. Testes de Usabilidade	17
10.1 Ronda de Concept (mínimo 4 utilizadores)	17
10.2 Ronda de Production (mínimo 4 utilizadores)	17
10.3 Ronda de Post-Production (mínimo 6 utilizadores)	17
11. Resultados Obtidos	18
12. Discussão dos Problemas Mais Importantes	19
12.1 Limites de quota da API Gemini	19
12.2 Dados desactualizados na VesselAPI	19
12.3 Correspondência incorrecta de fotos de navio	19
12.4 Deriva de consistência visual	19

12.5 Gestão de tempo nas rondas de usabilidade	20
13. Limitações Conhecidas	21
14. Conclusões	22
Anexo A — Application Concept Document (ACD)	23
1. Application Name and General Concept	23
2. Main Characteristics and Functionality	23
3. Target Audience	23
4. Business Model — Key Payable Feature	23
5–7. Navigation, Wireframes, Entity-Association	24
8. Firebase Architecture Plan	24
9. Technologies Used	24

1. Introdução

O **StabilityLoadingPlanner** é uma aplicação Android profissional desenvolvida para oficiais de marinha, cadetes e estudantes de engenharia naval, com o objectivo de planear operações de carregamento de carga e verificar a estabilidade de um navio em tempo real. A aplicação substitui o cálculo manual em papel — ainda comum na indústria marítima — por uma ferramenta digital que obtém dados técnicos do navio via API, calcula a estabilidade usando fórmulas de engenharia naval, e integra condições marítimas em tempo real no porto de partida.

O desenvolvimento seguiu a metodologia proposta na unidade curricular de Computação Móvel, centrada na experiência do utilizador (UX), estruturada em cinco fases sequenciais: *Proposal*, *Concept*, *Pre-Production*, *Production* e *Post-Production*. Cada fase produziu artefactos próprios — documento de conceito, wireframes, mapa de navegação, protótipo navegável, e finalmente a versão funcional completa — documentados ao longo deste relatório.

O modelo de negócio subjacente é *freemium*: todas as funcionalidades essenciais de planeamento de carga e cálculo de estabilidade estão disponíveis gratuitamente, enquanto a exportação de relatórios em PDF — uma funcionalidade legalmente relevante para oficiais que necessitam de arquivar documentação de estabilidade antes da partida — está reservada a uma subscrição PRO de €1/mês.

1.1 Público-alvo

O público principal são cadetes e oficiais de marinha em formação ou em exercício que necessitam de planear carga e verificar estabilidade. Secundariamente, estudantes de engenharia naval que precisem de uma ferramenta de cálculo de estabilidade, e oficiais de bordo que prefiram uma ferramenta digital rápida em vez de cálculos manuais.

2. Processo de Desenvolvimento

O desenvolvimento seguiu rigorosamente as quatro fases pós-proposta definidas no enunciado do projecto, cada uma com entregáveis próprios validados antes de avançar para a seguinte.

2.1 Fase de Concept

Nesta fase foram definidos o nome e conceito geral da aplicação, as entidades principais (User, Vessel, Tank/Hold, Cargo, Voyage, Marine Condition), o público-alvo, e a funcionalidade-chave de pagamento (exportação de relatório PDF por €1/mês). Foi produzido o *Application Concept Document* (ACD — Anexo A), os wireframes de todos os ecrãs (Secção 7), o mapa de navegação (Secção 8) e o diagrama de entidade-associação inicial (Secção 5). O conceito foi validado em aula com o docente, conforme exigido pelo enunciado como condição obrigatória para avançar de fase.

2.2 Fase de Pre-Production

Foi construído um protótipo mínimo navegável, com todos os ecrãs já existentes mas com dados simulados (hard-coded), permitindo validar o fluxo de navegação completo antes de implementar a lógica de negócio real. Foi também definido o esquema completo da base de dados Firestore (Secção 5) e os perfis de utilizador (FREE / PRO).

2.3 Fase de Production

Esta foi a fase de maior volume de trabalho técnico. Foram implementados: autenticação e persistência via Firebase (Authentication + Firestore), os clientes Retrofit para as cinco APIs externas utilizadas (VesselAPI, NGA World Port Index, Open-Meteo Marine, Wikimedia Commons/Wikipedia, e posteriormente Gemini AI), o motor de cálculo de estabilidade (GM/KG), o ecrã de planeamento de carga com distribuição dinâmica por porão, e a integração de fotos de navios. Foi também implementada a arquitectura de dados a dois níveis no Firestore — uma colecção *vessels/* partilhada entre todos os utilizadores (base de dados colaborativa de navios) e dados privados por utilizador em *users/{uid}/* — descrita em detalhe na Secção 5.

2.4 Fase de Post-Production

Na fase final foram acrescentados os detalhes que tornam a aplicação pronta para mercado: suporte multilingue completo (Português, Inglês e Espanhol) com selecção de idioma via diálogo no menu de definições, integração de IA generativa (Gemini) como método adicional de identificação de navios quando a base de dados e a API VesselAPI não têm informação suficiente, feedback visual consistente (snackbars, indicadores de progresso, avisos de sobrecarga), e três rondas de avaliação de usabilidade com utilizadores reais (Secção 10). Esta fase incluiu também um esforço significativo de consistência visual, descrito em detalhe na Secção 12.

3. Funcionalidades Implementadas

3.1 Funcionalidades Core (gratuitas)

Funcionalidade	Descrição
Pesquisa de Navio por IMO	Pesquisa em base de dados partilhada, depois VesselAPI, depois IA, com fallback para estimativas por tipo de navio
Estimação de Dimensões	Cálculo automático de LOA/Beam/DWT a partir do tipo de navio quando os dados de API não estão disponíveis
Planeamento de Carga	Distribuição dinâmica de carga por porão, com selecção de tipo de carga e factor VCG
Cálculo de Estabilidade	GM e KG calculados em tempo real usando fórmulas de arquitectura naval ($KB + BM/V$)
Condições Marítimas	Altura e período de ondas, temperatura da água em tempo real via Open-Meteo Marine API
Pesquisa de Portos	Mais de 3700 portos mundiais via NGA World Port Index, pesquisáveis por nome ou país
Fotos de Navios	Obtidas automaticamente via Wikimedia Commons / Wikipedia a partir do número IMO ou nome
Autenticação	Login/registo via Firebase Authentication (email/password)
Multilingue	Português, Inglês e Espanhol — selecção automática pela língua do dispositivo ou manual via menu
Ajuda e Sobre	Guia de utilização passo-a-passo e identificação do autor

3.2 Funcionalidade PRO (€1/mês)

A exportação de relatório em PDF é a funcionalidade-chave do modelo de negócio freemium. O relatório inclui dados do navio, plano de carga distribuído por porão, valores de GM/KG calculados, e estado GO/NO-GO de estabilidade — documentação que a regulamentação marítima exige antes da partida. Num cenário de mercado global com cerca de 1,7 milhões de marítimos activos, mais estudantes navais, agentes portuários e profissionais de logística, uma conversão freemium para esta funcionalidade documental é um objectivo realista.

3.3 Funcionalidade Extra — Inteligência Artificial

Foi integrada a API Gemini (Google AI) como método adicional de identificação de navios: quando a base de dados partilhada e a VesselAPI não devolvem dados suficientes para um determinado IMO, a aplicação envia um pedido estruturado ao modelo `gemini-2.0-flash`, que tenta identificar o navio e devolver as suas dimensões técnicas em formato JSON. Esta integração e as suas limitações em produção são discutidas em detalhe na Secção 12.

4. Arquitectura e Tecnologias

A aplicação segue o padrão arquitectural **MVVM** (Model-View-ViewModel), idiomático em Jetpack Compose. Dois ViewModels centrais — `VesselViewModel` e `AuthViewModel` — expõem estado observável (`mutableStateOf`) consumido directamente pelos ecrãs Compose, eliminando a necessidade de gestão manual de estado na camada de UI.

4.1 Stack Tecnológico

Camada	Tecnologia
Linguagem	Kotlin
UI	Jetpack Compose + Material 3
Arquitectura	MVVM (ViewModel + mutableStateOf)
Navegação	Jetpack Navigation Compose
Rede / HTTP	Retrofit 2 + OkHttp 3
Carregamento de Imagens	Coil
Autenticação	Firebase Authentication (Email/Password)
Base de Dados	Firebase Firestore (Cloud, NoSQL)
Geração de PDF	Android PdfDocument + MediaStore API
IA Generativa	Google Gemini API (gemini-2.0-flash)

4.2 APIs Externas

API	Finalidade	Autenticação
VesselAPI	Dados técnicos de navio por IMO	Bearer token (tier gratuito)
NGA World Port Index	+3700 portos mundiais	Nenhuma (pública)
Open-Meteo Marine	Condições marítimas em tempo real	Nenhuma (pública)
Wikimedia Commons / Wikipedia	Fotos de navios	User-Agent requerido
Google Gemini	Identificação de navio via IA	API Key

4.3 Estratégia de Estado: Privado vs. Partilhado

A decisão arquitectural mais relevante do projecto foi separar o estado da aplicação em dois níveis no Firestore. A colecção `vessels/{imo}` é **partilhada** — todos os utilizadores autenticados podem ler e escrever. Quando um utilizador corrige dimensões estimadas de um navio através do diálogo de edição, essa correcção fica disponível para todos os futuros utilizadores que pesquisarem o mesmo IMO, criando uma base de dados de navios construída colaborativamente que melhora com o uso. Os dados de histórico de pesquisa, navios guardados e viagens, dentro de `users/{uid}/`, são **privados** a cada utilizador. Esta separação está detalhada no diagrama de entidade-associação (Secção 5).

5. Diagrama de Entidade-Associação e Esquema da Base de Dados

A aplicação usa Firebase Firestore, uma base de dados NoSQL orientada a documentos. O diagrama seguinte mostra as seis colecções principais, os seus campos, e as relações entre elas — distinguindo dados partilhados (toda a comunidade de utilizadores) de dados privados (por utilizador autenticado).



Figura 5.1 — Diagrama de Entidade-Associação e tabela de relações

O esquema completo das colecções Firestore, incluindo sub-colecções e tipos de dados, está representado em árvore na Figura 5.2 — equivalente ao esquema de base de dados (sem dados) exigido para este relatório.



Figura 5.2 — Esquema completo das colecções Firestore

6. Diagrama de Classes UML

A aplicação segue o padrão MVVM com duas camadas principais de classes: as *data classes* que modelam as entidades de negócio, e os ViewModels que orquestram o estado da aplicação e dependem das interfaces de serviço Retrofit para comunicação com as APIs externas. Os nomes de campos e tipos reflectem directamente a implementação Kotlin.

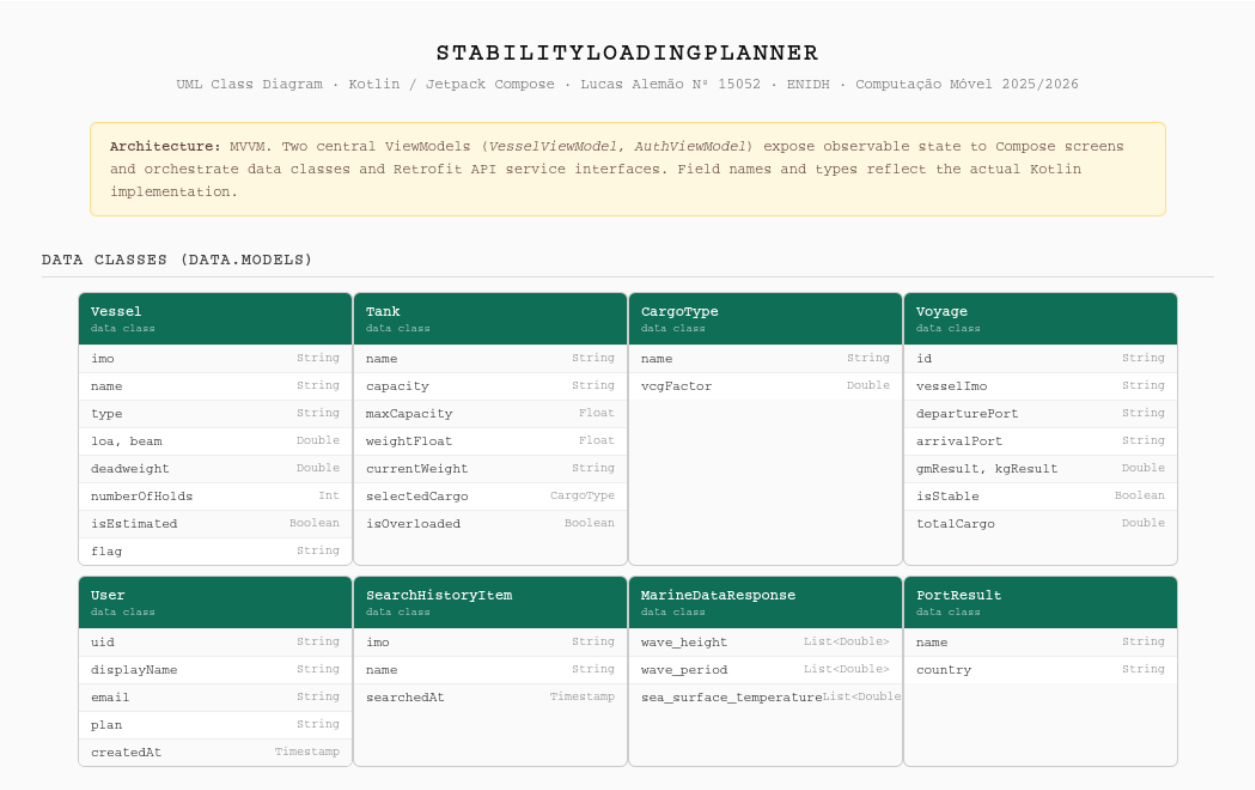


Figura 6.1 — Data classes do pacote data.models

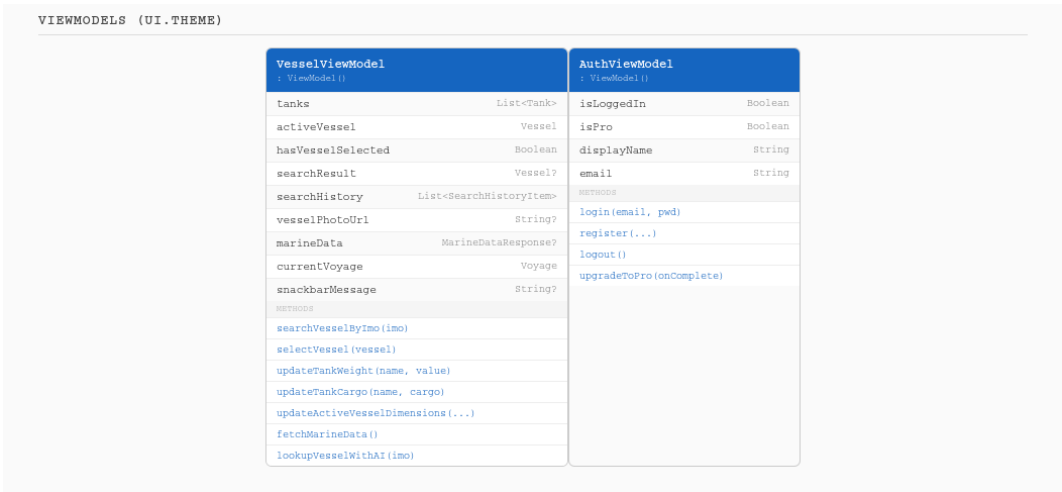


Figura 6.2 — ViewModels (VesselViewModel e AuthViewModel)

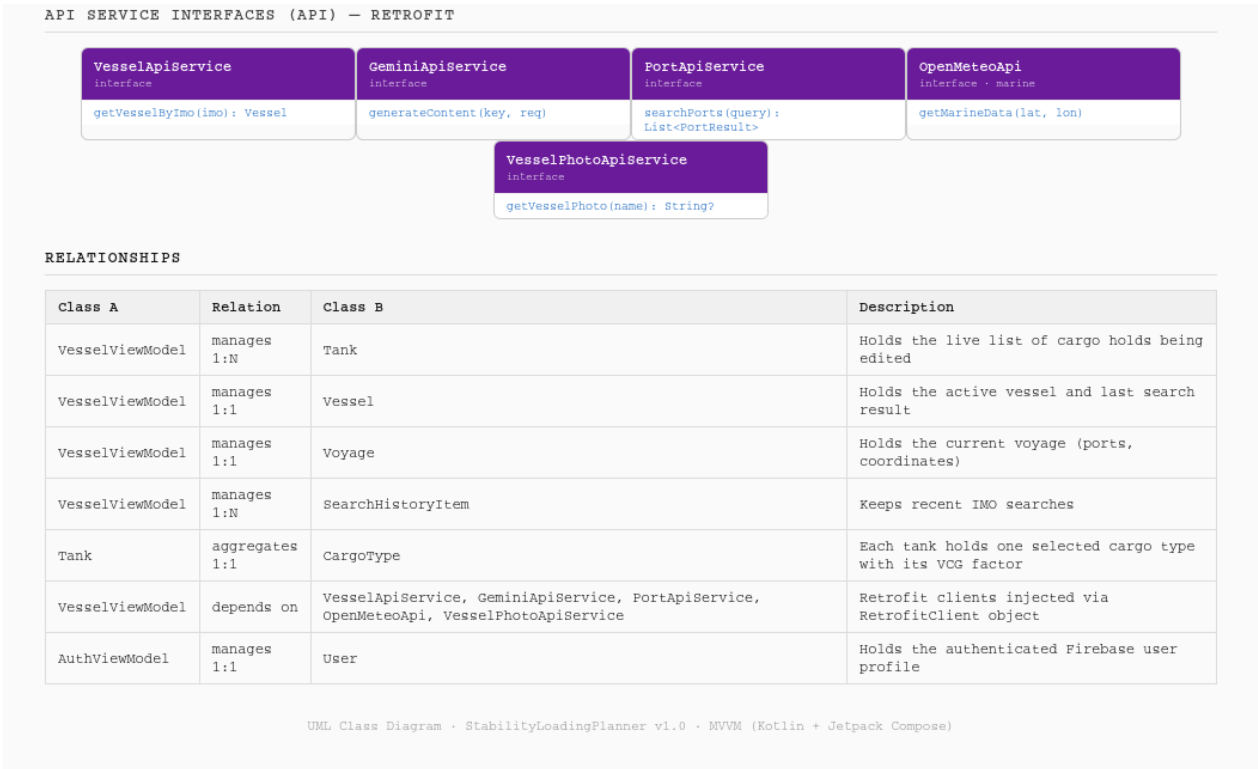


Figura 6.3 — Interfaces de serviço API (Retrofit) e tabela de relações entre classes

7. Wireframes e Mock-ups Iniciais

Os wireframes de baixa fidelidade de todos os 12 ecrãs foram produzidos na fase de Concept, focados em estrutura de conteúdo e fluxo, sem detalhes visuais definitivos — conforme a definição de wireframe do enunciado.

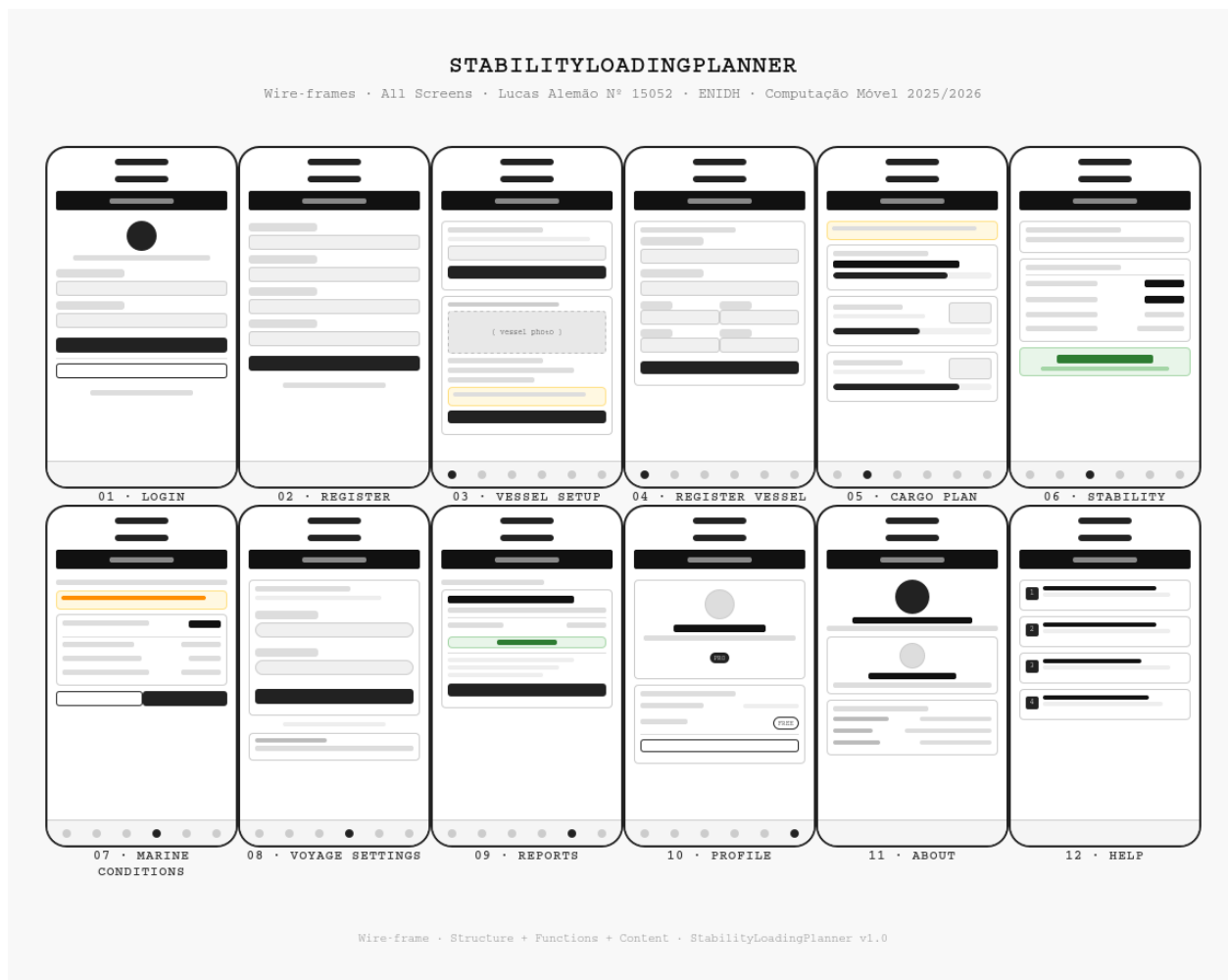


Figura 7.1 — Wireframes de baixa fidelidade, todos os 12 ecrãs

7.1 Nota sobre a fase de Mock-up

A definição da linguagem visual de alta fidelidade (cores, tipografia, espaçamento) não foi produzida como um documento de mock-up estático separado. Em vez disso, foi definida directamente como design tokens no tema Jetpack Compose da aplicação (Color.kt, Theme.kt, Type.kt — paleta industrial-marítima com cores IndustrialPrimary, IndustrialSurface e IndustrialBackground), e validada iterativamente directamente nos ecrãs funcionais à medida que eram implementados. Esta abordagem fundiu as fases de mock-up e prototipagem, com a vantagem de cada decisão visual ser imediatamente testável em código real, mas com a contrapartida de não existir um artefacto de mock-up estático isolado para incluir aqui como documento independente.

8. Mapa de Navegação

O mapa seguinte documenta todos os ecrãs da aplicação e as transições de navegação entre eles, incluindo o fluxo de autenticação, o fluxo principal de trabalho (Vessel Setup → Cargo Plan → Stability → Marine Conditions → Reports), e o acesso a ecrãs utilitários via o menu de três pontos e a barra de navegação inferior.

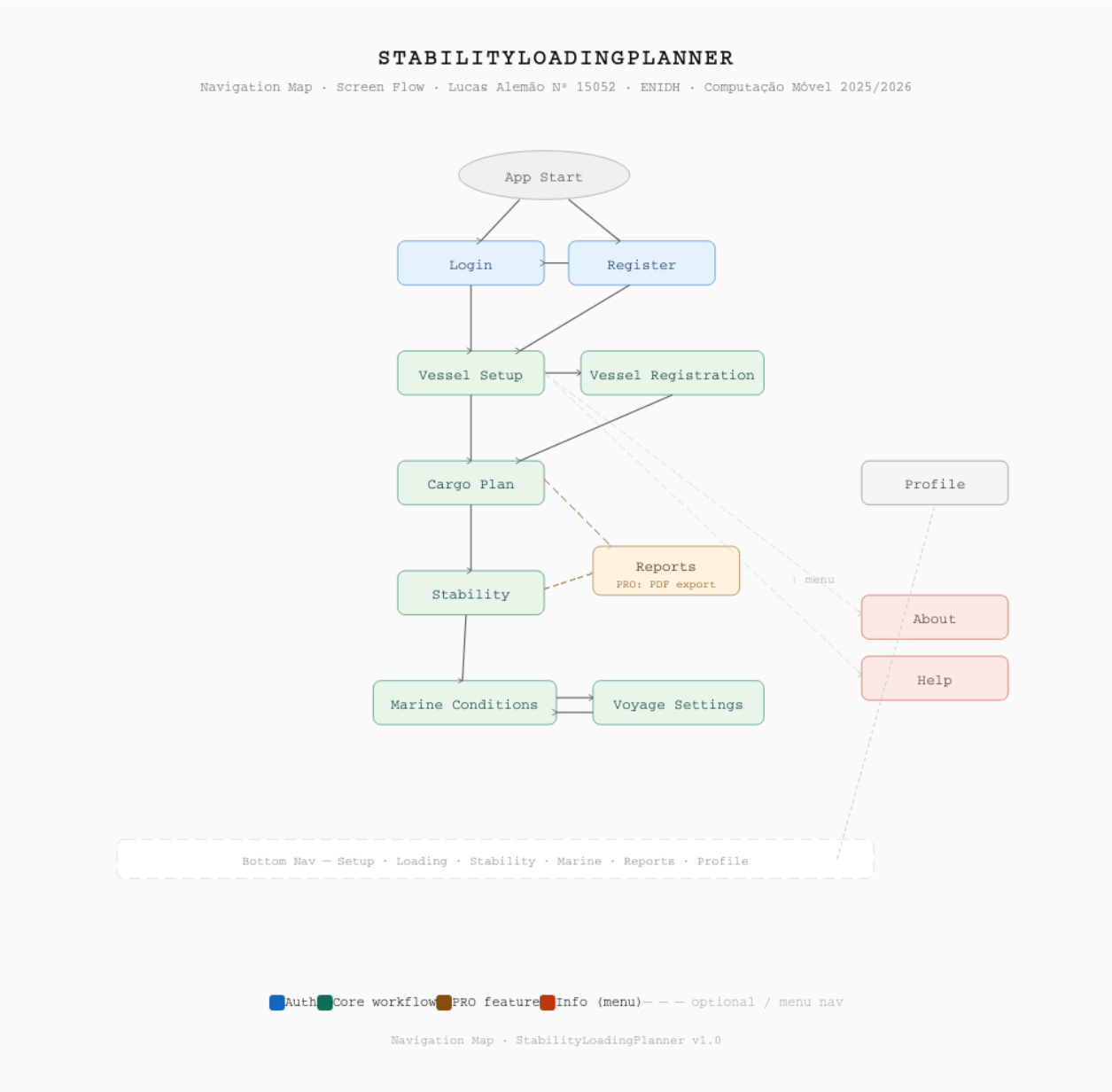


Figura 8.1 — Mapa de navegação completo

9. Capturas de Ecrã Finais

A Figura 9.1 mostra o ecrã de Vessel Setup em funcionamento real, com o navio FEDERAL SPRUCE (IMO 9610468) identificado por número IMO, fotografia correctamente correspondida, e dimensões técnicas estimadas apresentadas de forma clara.

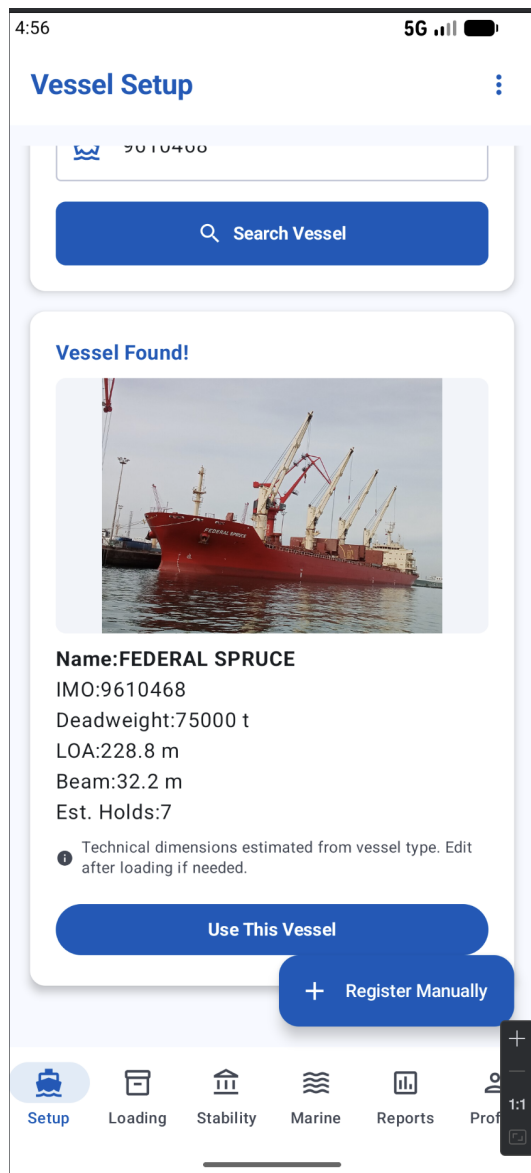


Figura 9.1 — Ecrã Vessel Setup, navio FEDERAL SPRUCE identificado por IMO

As figuras seguintes documentam os restantes ecrãs principais da aplicação em funcionamento, incluindo o ecrã de autenticação, o fluxo de planeamento e estabilidade para o mesmo navio (FEDERAL SPRUCE), condições marítimas, exportação de relatório PRO, perfil de utilizador, e o diálogo de selecção de idioma.

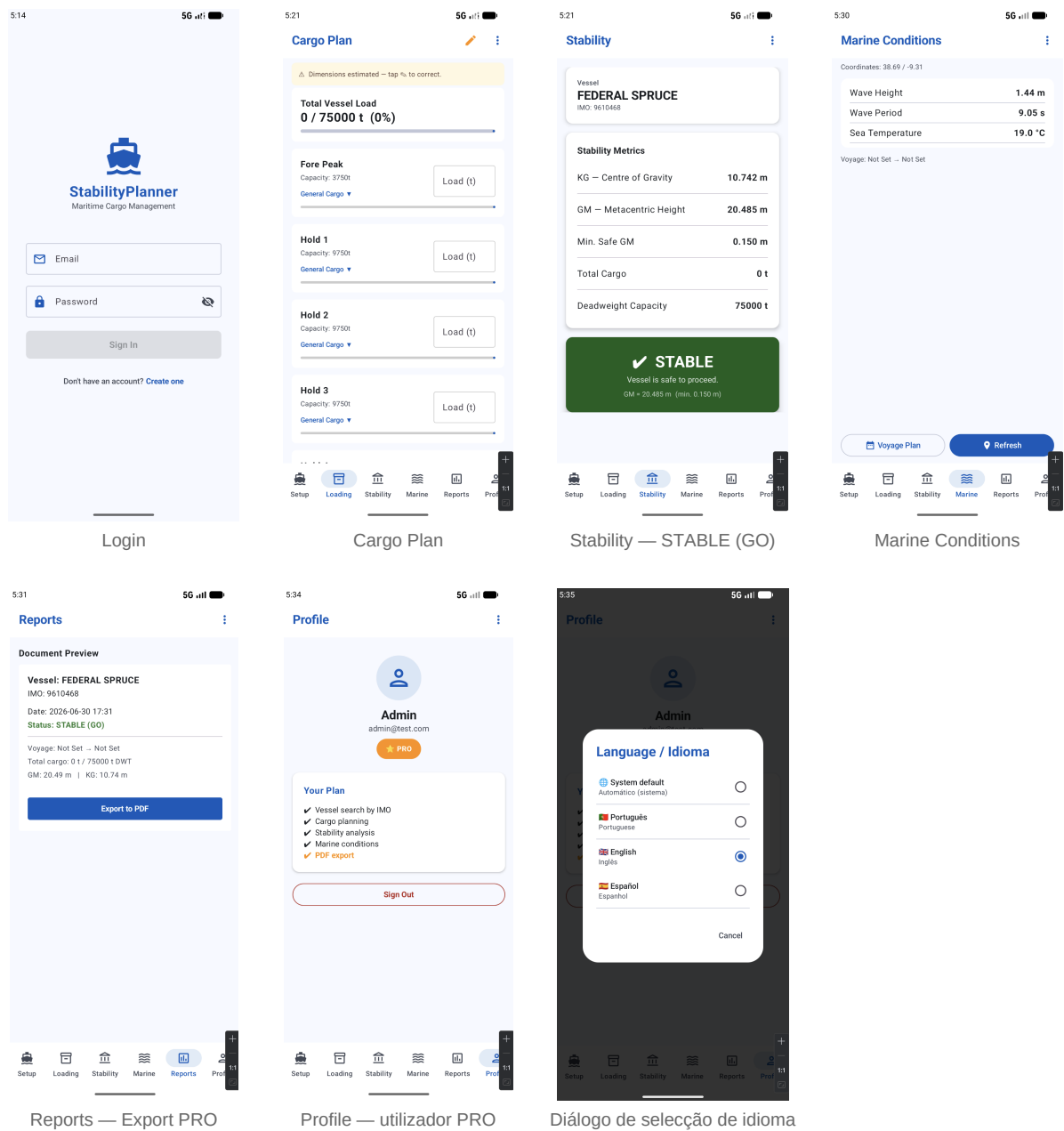


Figura 9.2 — Ecrãs principais: Login, Cargo Plan, Stability, Marine Conditions, Reports, Profile, e selecção de idioma

10. Testes de Usabilidade

Conforme a metodologia do enunciado, a avaliação de usabilidade foi planeada em três rondas distintas, com questionários e objectivos diferentes em cada fase.

10.1 Ronda de Concept (mínimo 4 utilizadores)

Pendente. Esta ronda — validação do conceito e wireframes junto de utilizadores externos à equipa, antes da existência de uma app funcional — não foi concluída até à data deste relatório. A validação do conceito em si foi feita directamente com o docente da unidade curricular, conforme exigido como condição obrigatória de avanço de fase. O inquérito formal aos utilizadores externos será acrescentado a este relatório assim que recolhido.

10.2 Ronda de Production (mínimo 4 utilizadores)

Foram conduzidos testes de usabilidade com 3 utilizadores reais, usando a versão funcional da aplicação. Cada utilizador realizou de forma independente as tarefas de pesquisar um navio por IMO, criar um plano de carga, e verificar o resultado de estabilidade.

Utilizador	Completoou as tarefas?	Observações
User 1	Sim, sem ajuda	Hesitação breve ao navegar de Cargo Plan para Stability — esperava transição automática
User 2	Sim, sem ajuda	Mesma hesitação que User 1 — confirmou o mesmo ponto de fricção
User 3	Sim, sem ajuda	Navegou para Stability sem hesitar; identificou que as dimensões do navio são estimadas e não correspondem exactamente às reais

Os três utilizadores classificaram a aplicação como positiva no final do teste. Foi identificado um ponto de fricção consistente — a falta de uma transição ou indicação clara do próximo passo depois de preencher o plano de carga — que foi corrigido ainda durante o desenvolvimento com a adição de um botão de chamada-à-acção ("View Stability Analysis") no final da lista do Cargo Plan, visível assim que existe carga lançada em pelo menos um porão.

10.3 Ronda de Post-Production (mínimo 6 utilizadores)

Pendente. Esta ronda final, que repete o questionário da fase de Production com 6 utilizadores adicionais distintos dos anteriores, está agendada para ser concluída antes da discussão oral.

11. Resultados Obtidos

No final da fase de Post-Production, a aplicação encontra-se funcional e estável, cumprindo a generalidade dos requisitos mínimos definidos no enunciado:

- Aplicação com mais de 4 ecrãs principais (Setup, Cargo Plan, Stability, Marine Conditions, Reports, Profile), navegação completa e consistente entre todos eles.
- Autenticação de utilizadores via Firebase Authentication, com estado da aplicação guardado localmente e remotamente (Firestore), incluindo dados privados e partilhados.
- Modelo de negócio implementado: subscrição PRO simulada a €1/mês, com funcionalidade de exportação PDF como diferenciador.
- Suporte multilingue para Português, Inglês e Espanhol, com selecção automática pela língua do sistema ou manual via diálogo.
- Pelo menos uma animação (transições `AnimatedVisibility` no resultado de pesquisa de navio e na confirmação de upgrade PRO) e imagens de tamanho pequeno e grande (ícones e fotos de navio).
- Cinco integrações de API externas funcionais em produção: VesselAPI, NGA World Port Index, Open-Meteo Marine, Wikimedia/Wikipedia, e Google Gemini.
- Motor de cálculo de estabilidade naval (GM/KG) funcional e validado manualmente com valores de teste.
- Feedback positivo dos três utilizadores que testaram a aplicação na ronda de Production, com um problema de usabilidade identificado e corrigido ainda durante o desenvolvimento.

12. Discussão dos Problemas Mais Importantes

Esta secção documenta, de forma honesta, os problemas técnicos mais significativos encontrados durante o desenvolvimento e a forma como foram resolvidos ou mitigados.

12.1 Limites de quota da API Gemini

Durante a fase final de testes, a chave de API Gemini associada ao projecto atingiu o limite diário de quota do tier gratuito (erro HTTP 429, *RESOURCE_EXHAUSTED*), resultado de múltiplos testes consecutivos durante o desenvolvimento. Foi também identificado um erro HTTP 404 ao tentar usar o modelo *gemini-1.5-flash*, que não está disponível para esta chave de API — o modelo correcto e disponível é o *gemini-2.0-flash*, originalmente configurado. Este episódio reforçou a importância de validar a disponibilidade exacta de modelos por chave de API antes de assumir compatibilidade entre versões, e de não depender de uma única chave de testes partilhada durante iterações intensivas de desenvolvimento.

12.2 Dados desactualizados na VesselAPI

Foi identificado um caso em que a VesselAPI devolveu o nome antigo de um navio (IMO 9641314, devolvido como "LAGARFOSS") quando o nome actual e publicamente verificável é "ATLANTICO". Navios mudam de nome ao longo da sua vida operacional e as APIs de terceiros nem sempre reflectem essa mudança em tempo real. A aplicação mitiga este problema através do diálogo de edição manual, que permite a qualquer utilizador corrigir os dados — correcção que fica disponível para todos os futuros utilizadores da mesma base de dados partilhada (Secção 4.3).

12.3 Correspondência incorrecta de fotos de navio

A pesquisa de fotos via Wikipedia/Wikimedia Commons usa o nome do navio como termo de pesquisa, o que ocasionalmente devolve resultados incorrectos quando o nome coincide com outra entidade — por exemplo, o navio "CORVO" devolveu uma fotografia da ilha do Corvo, e o navio "JAIME S" devolveu uma fotografia de uma instalação terrestre (por exemplo, o caso do navio JAIME S, que devolveu uma imagem de uma instalação terrestre de comunicações em vez de um navio). Esta é uma limitação conhecida e documentada (Secção 13), sem solução completa sem recurso a uma API paga e especializada em fotografia de navios.

12.4 Deriva de consistência visual

Durante iterações rápidas de funcionalidades (integração de IA, condições marítimas), alguns ecrãs foram reescritos usando directamente as cores padrão do Material 3 (`MaterialTheme.colorScheme`) em vez dos tokens de tema industrial definidos para a aplicação (`IndustrialPrimary`, `IndustrialSurface`), resultando numa barra de navegação inferior e ecrãs com cores inconsistentes com o resto da aplicação. O problema foi identificado através de inspecção visual directa e corrigido restaurando o uso consistente dos tokens de tema em todos os ecrãs e componentes partilhados (`Components.kt`). Esta experiência reforçou a importância de centralizar definições de estilo em componentes reutilizáveis (`ExactBottomNav`, `AppMenuActions`) em vez de duplicar definições de cor em cada ecrã individualmente.

12.5 Gestão de tempo nas rondas de usabilidade

Como documentado na Secção 10, as rondas de Concept e Post-Production não foram concluídas até à data deste relatório, ao contrário da ronda de Production. Esta foi uma falha de planeamento temporal — a estrutura de três rondas distintas com mínimos de participantes diferentes (4, 4 e 6 utilizadores) só foi plenamente compreendida numa fase tardia do desenvolvimento — e está identificada como acção pendente antes da discussão oral.

13. Limitações Conhecidas

- **Precisão das estimativas:** quando nem a base de dados partilhada, nem a VesselAPI, nem a IA Gemini têm dados suficientes para um navio, a aplicação recorre a fórmulas de estimativa genéricas por tipo de navio, que podem divergir significativamente das dimensões reais. O utilizador pode sempre corrigir manualmente.
- **Fotos de navio incorrectas:** a pesquisa por nome no Wikipedia/Wikimedia Commons pode devolver imagens não relacionadas (Secção 12.3), sem solução completa sem uma API paga especializada.
- **Ecrã About sem fotografia do autor:** por decisão consciente, o ecrã About não inclui fotografia do autor, apenas identificação textual (nome e número de aluno) — o enunciado pede explicitamente identificação e fotografia, pelo que esta omissão é assumida como risco de perda de pontuação nesse requisito específico.
- **Rondas de usabilidade incompletas:** as rondas de Concept (4 utilizadores) e Post-Production (6 utilizadores) ainda não foram realizadas à data deste relatório (Secção 10).
- **Quota gratuita da API Gemini:** a integração de IA está sujeita aos limites do tier gratuito da API Gemini, que podem ser esgotados com uso intensivo, fazendo a aplicação reverter para estimativas locais.

14. Conclusões

O StabilityLoadingPlanner cumpre o objectivo central proposto: substituir o cálculo manual de estabilidade de carga por uma ferramenta digital funcional, integrando dados reais de múltiplas fontes externas num único fluxo de trabalho coerente. O desenvolvimento seguiu de forma estruturada as cinco fases da metodologia centrada no utilizador, com validação de conceito junto do docente e testes de usabilidade reais que já produziram pelo menos uma correcção concreta à interface.

Os problemas técnicos encontrados — limites de quota de API, dados desactualizados de terceiros, correspondência incorrecta de imagens, e deriva de consistência visual durante iteração rápida — são representativos dos desafios reais de integrar múltiplos serviços externos numa aplicação móvel, e foram documentados com transparência na Secção 12 como parte do valor formativo deste projecto.

Ficam pendentes, e identificadas como prioridade antes da discussão oral, as duas rondas de usabilidade em falta (Concept e Post-Production) e a obtenção de uma chave de API Gemini com quota disponível para demonstração ao vivo da funcionalidade de IA. O código fonte, a documentação de processo, e este relatório constituem, em conjunto, um registo completo e honesto do trabalho realizado.

Anexo A — Application Concept Document (ACD)

1. Application Name and General Concept

Name: StabilityLoadingPlanner

Concept: A professional Android application designed for maritime officers, cadets, and naval engineering students to plan cargo loading operations and verify vessel stability. The app replaces manual paper-based calculations with a real-time digital tool that fetches vessel data via API, calculates stability using certified naval engineering formulas, and integrates live marine weather at the departure port.

The application targets the maritime industry, a field where stability calculations are legally required before departure and errors can have catastrophic consequences.

2. Main Characteristics and Functionality

Entities:

- **User** — authenticated account with FREE or PRO plan
- **Vessel** — ship identified by IMO number, with technical particulars (LOA, beam, DWT, KG, holds)
- **Tank/Hold** — cargo compartment with capacity, weight, and cargo type
- **Cargo** — type of goods loaded (general cargo, grain, steel coils, containers), each with a VCG factor
- **Voyage** — route between two ports with departure/arrival data
- **Marine Condition** — real-time sea state at the voyage coordinates

Core Functionality:

- Vessel Search by IMO — queries VesselAPI for vessel identification and dimensions; auto-estimates missing technical data using naval architecture formulas
- Cargo Plan — user distributes cargo across holds; app calculates total load vs deadweight capacity
- Stability Analysis — computes GM (metacentric height) and KG (centre of gravity) using KB + BM/V formulas; classifies vessel as STABLE ($GM \geq 0.15m$) or UNSTABLE
- Marine Conditions — fetches live wave height, period, sea temperature from Open-Meteo Marine API
- Port Database — 3,700+ global ports via NGA World Port Index; searchable by name or browseable by country
- Vessel Photos — auto-fetched from Wikimedia Commons using the IMO number as search key
- Report Export — PDF report with vessel data, cargo plan, and stability analysis saved to Downloads (PRO)

3. Target Audience

Primary: Maritime cadets and officers studying or practicing cargo planning. **Secondary:** Naval engineering students needing a stability calculation tool. **Tertiary:** Ship officers needing a quick digital tool instead of manual calculations.

4. Business Model — Key Payable Feature

Model: Freemium — €1/month PRO subscription

Free tier: All core features — vessel search, cargo planning, stability calculation, marine conditions, port search, vessel photos.

PRO feature: PDF Export — the ability to generate and download a legally-formatted stability and loading report as a PDF, including vessel particulars, cargo distribution, GM/KG values, and GO/NO-GO status.

10 million user scenario: in a global maritime industry with 1.7 million seafarers worldwide, and a broader market of maritime students, port agents, and logistics professionals, a €1/month PDF export feature for those needing documented reports is a realistic freemium conversion target.

5–7. Navigation, Wireframes, Entity-Association

Ver Secções 8, 7 e 5 deste relatório, respectivamente, para os diagramas completos que desenvolvem estes pontos do ACD original.

8. Firebase Architecture Plan

Key architectural decision: the `vessels/` collection is shared across all users. When a user corrects estimated vessel dimensions using the edit dialog, those corrections are saved to `vessels/{imo}` and become available to all future users searching the same IMO. This creates a community-sourced vessel database that improves with each user.

9. Technologies Used

Technology	Purpose	Status
Kotlin + Jetpack Compose	Android UI	Implemented
Retrofit 2 + OkHttp	HTTP API clients	Implemented
Coil	Image loading	Implemented
VesselAPI	Vessel data by IMO	Implemented
NGA World Port Index	Global port database	Implemented
Open-Meteo Marine	Live sea conditions	Implemented
Wikimedia Commons API	Vessel photos	Implemented
Firebase Auth	User authentication	Implemented
Firebase Firestore	Cloud state storage	Implemented
Google Gemini API	AI vessel identification	Implemented

Nota: a versão original do ACD (fase de Concept) listava Firebase Auth e Firestore como "Planned"; foram entretanto implementados na fase de Production, tal como a integração com a API Gemini, não prevista no ACD inicial e aprovada posteriormente como funcionalidade extra.